International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

HTTP Analysis Using Wireshark: Text Traffic

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 08/09/2026

EXECUTIVE SUMMARY

This practical exercise focused on the capture, preservation, and forensic analysis of plaintext HTTP network traffic within a controlled Kali Linux environment. A local Apache web server was configured to host a webpage containing the trainee's identification details, including the name Kafayat Animashawun and registration number 2025/FWSD/11317. An HTTP session was generated against the local web server and captured on the loopback interface using TShark/Wireshark. The resulting PCAPNG file was preserved as forensic evidence and subjected to integrity verification using SHA-256 hashing. The captured traffic was then examined to identify the TCP three-way handshake, HTTP GET request, HTTP 200 OK response, HTTP headers, transmitted webpage content, TCP stream, and connection termination sequence. The exercise demonstrates how plaintext HTTP communications can expose network and application-layer information to a forensic analyst and highlights the importance of proper evidence preservation and analysis procedures.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to develop practical skills in network forensic investigation through the controlled capture and analysis of plaintext HTTP traffic. The exercise provides an opportunity to observe how a TCP connection is established, how HTTP requests and responses are exchanged, and how the resulting network packets can be examined to reconstruct a communication session. It also introduces fundamental forensic evidence-handling practices, including preservation of the original packet

capture, creation of a working copy, and verification of evidence integrity using cryptographic hashing.

OBJECTIVES OF THE ASSIGNMENT

The objectives of this practical assignment are to:

1. Configure a local Apache web server to generate controlled plaintext HTTP traffic.
2. Create and access a webpage containing identifiable trainee and laboratory information.
3. Verify that the HTTP service is listening on TCP port 80.
4. Capture the HTTP session using TShark/Wireshark on the loopback interface.
5. Preserve the original PCAPNG capture and create a separate working copy for analysis.
6. Calculate and compare SHA-256 hashes to verify the integrity of the captured evidence.
7. Identify and analyse the TCP three-way handshake consisting of SYN, SYN-ACK, and ACK packets.
8. Examine the HTTP GET request and associated request headers.
9. Analyse the HTTP 200 OK response, response headers, and returned webpage content.
10. Reconstruct the communication using the Follow TCP Stream functionality.
11. Examine the TCP connection termination sequence.
12. Document the findings using screenshots, packet details, and forensic observations.

METHODOLOGY OF THE ASSIGNMENT

The practical was conducted in a controlled Kali Linux virtual machine environment. First, the required networking and forensic analysis tools, including Apache2, cURL, Wireshark, and TShark, were installed and verified. The Apache web server was then enabled and configured to listen on TCP port 80. A local HTML webpage was created containing the trainee's name, registration number, and laboratory identifier. The webpage was tested using cURL to confirm successful HTTP communication and to observe the request and response headers.

Following successful verification, TShark was configured to capture TCP traffic associated with port 80 on the loopback interface. A controlled HTTP request was generated using cURL while the capture was active. After the transaction was completed, the capture was stopped and the resulting PCAPNG file was validated using capture-file analysis tools. The original evidence file was preserved and copied to a separate working directory for analysis. SHA-256 hashes were calculated for both files to confirm that the working copy remained identical to the original evidence.

The captured packets were subsequently analyzed using Wireshark/TShark. The investigation examined the TCP connection establishment sequence, including the SYN, SYN-ACK, and ACK packets, followed by the HTTP GET request and server's HTTP 200 OK response. Packet-level information such as source and destination IP addresses, TCP ports, sequence numbers, acknowledgement numbers, timestamps, and HTTP headers was examined. The HTTP conversation was also reconstructed using Follow TCP Stream, and the TCP connection termination packets were reviewed to establish how the session ended. Screenshots were collected throughout the process to provide supporting evidence for the findings documented in this report.

TOOLS AND RESOURCES USED

The following tools and resources were used during the practical exercise:

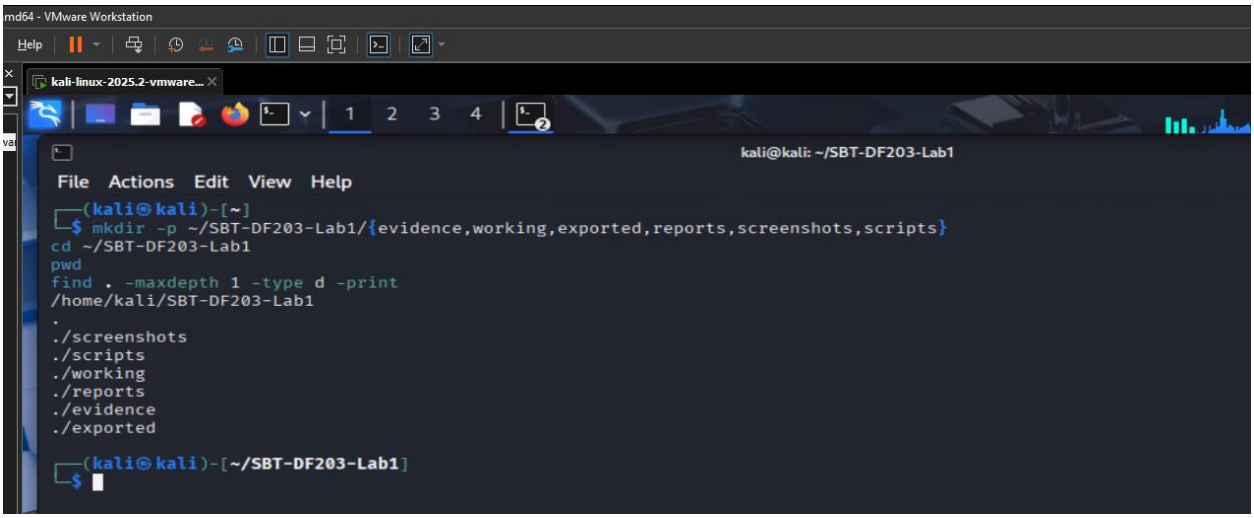
S/N	Tool / Resource	Purpose / Application
1	**Kali Linux**	Provided the controlled operating environment for conducting the network forensic investigation.
2	**Apache2**	Used to host the controlled local HTTP webpage and generate plaintext HTTP traffic over TCP port 80.
3	**cURL**	Used to generate HTTP GET requests and verify the HTTP request and response between the client and local web server.
4	**Wireshark**	Used to inspect and analyze captured packets, including TCP handshake, HTTP requests/responses, TCP streams, and connection termination.

S/N	Tool / Resource	Purpose / Application
5	**TShark**	Used to capture the HTTP network traffic and save the captured packets as a PCAPNG evidence file.
6	**TCP/IP Protocol Suite**	Provided the underlying communication protocols examined during the investigation, particularly TCP.
7	**HTTP Protocol**	Application-layer protocol analyzed to demonstrate plaintext request headers, response headers, and webpage content.
8	**Loopback Interface (lo)**	Used to capture communication between the local HTTP client and Apache web server on the same Kali Linux system.
9	**PCAPNG Evidence File**	Served as the primary network forensic artefact containing the captured HTTP session.
10	**SHA-256 Hashing**	Used to verify the integrity and consistency of the original capture and its working copy.
11	**Kali Linux Terminal**	Used to execute commands for service configuration, traffic generation, packet capture, evidence preservation, and analysis.
12	**Local HTML Webpage**	Served as the controlled source of HTTP content containing the trainee's name, registration number, and laboratory identifier.

Creation of the Lab Working Environment

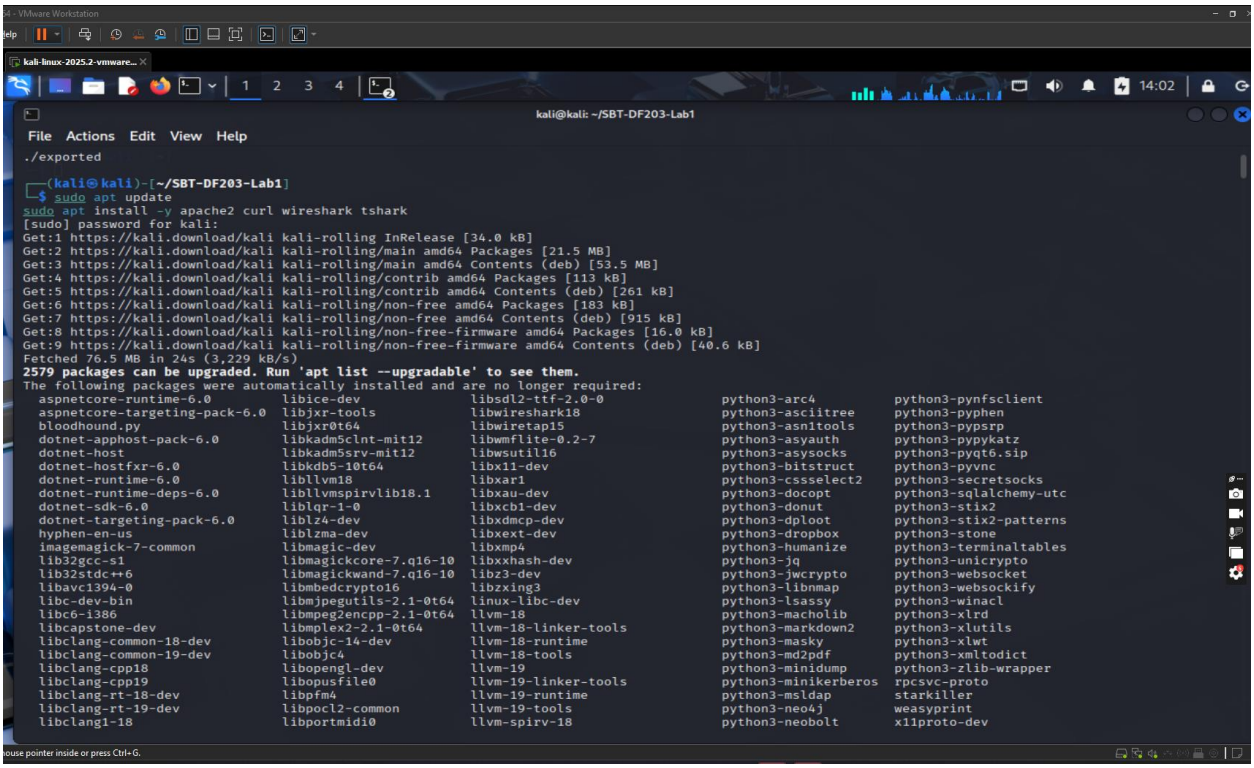
The designated working directory for the SBT-DF203 Lab 1 practical was created on the Kali Linux forensic workstation. The directory was structured into separate folders for evidence The practical exercise commenced with the creation of a dedicated working directory for the SBT-DF203 Lab 1 investigation. A structured directory named SBT-DF203-Lab1 was created within the Kali Linux home directory, with separate subdirectories for evidence, working, exported, reports, screenshots, and scripts. This structure was established to support organized forensic evidence handling throughout the investigation. The evidence directory was designated for original packet capture files, while the working directory was intended for analysis copies. The reports directory was used to store investigative outputs such as command results and case information, while

the screenshots directory was reserved for visual documentation of the practical activities. The creation of these directories before traffic generation provided a controlled workspace for separating original evidence from subsequent analysis and documentation activities.



Installation of Required Tools

Following the preparation of the forensic workspace, the required networking and forensic analysis tools were installed and their versions verified. The Kali Linux environment was updated using the Advanced Package Tool, after which Apache2, curl, Wireshark and TShark were installed. The subsequent version checks confirmed the availability of Apache version 2.4.68, curl version 8.21.0, Wireshark version 4.6.6 and TShark version 4.6.6. These tools provided the necessary capabilities for the practical exercise: Apache was used as the local HTTP server, curl was used to generate controlled HTTP traffic, while Wireshark and TShark were available for packet capture and protocol-level analysis. The tool verification established that the Kali Linux virtual machine had the required software environment before evidence acquisition commenced.



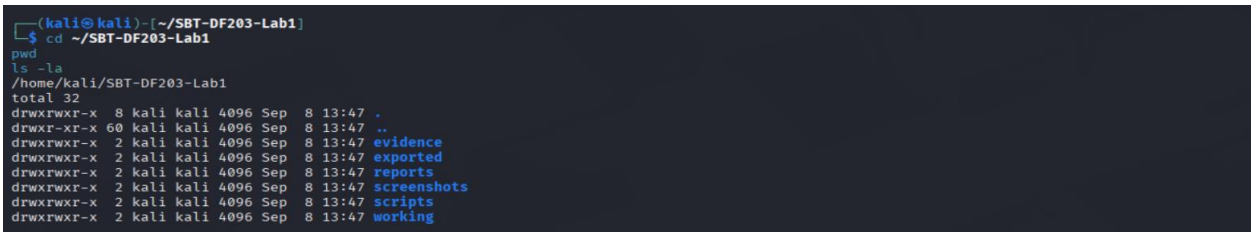
Verification of Required Lab Tools

The screenshot shows the successful verification of the required tools for the SBT-DF203 Lab 1 environment. Apache HTTP Server, cURL, Wireshark, and TShark were executed from the Kali Linux terminal to confirm that they were installed and functioning correctly. The output displays the installed versions of each tool, confirming that the forensic analysis and network investigation environment was successfully prepared.

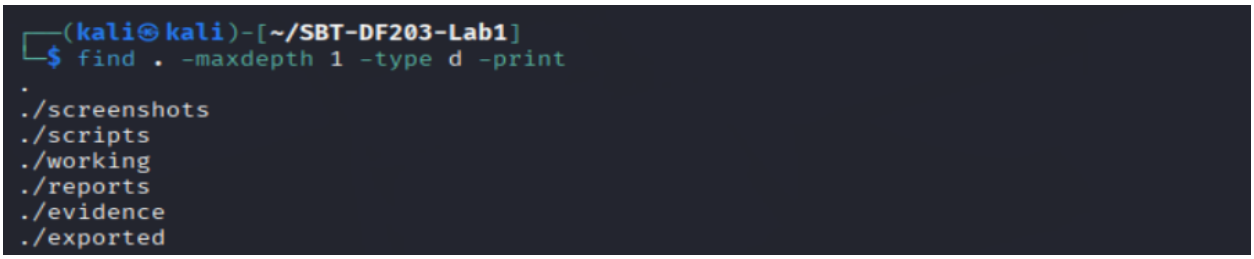
Verification of the Lab Directory Structure

The directory structure was subsequently verified from the SBT-DF203-Lab1 working directory. The output confirmed that the six designated subdirectories were present and accessible: evidence, working, exported, reports, screenshots, and scripts. At this stage, the evidence and analysis directories were empty, indicating that packet capture and analytical outputs had not yet been generated. This verification was important because it

established the initial state of the laboratory workspace before evidence acquisition and provided a clear organizational baseline for subsequent forensic activities.



The screenshot shows the structured working directory created for the SBT-DF203 Lab 1 investigation. The workspace contains separate directories for evidence, working files, exported packet-analysis results, reports, screenshots, and scripts. This structure was established to support organized handling of forensic artefacts and to maintain a clear distinction between the original captured evidence and files generated during analysis. The separation of these directories helps preserve the integrity of the original network capture while allowing analysis outputs to be generated without modifying the source evidence.



Case Information and Evidence Identification

A case information record was created within the reports directory to establish the basic identification and scope of the practical investigation. The record identified the case as SBT-DF203-Lab1-Kafayat-Animashawun, documented the trainee's name as Kafayat Animashawun, recorded the practical date as 08 September 2026, and identified the analysis environment as a Kali Linux virtual machine. The purpose of the exercise was also documented as controlled plaintext HTTP traffic analysis. Maintaining this information in a dedicated report file provided a basic evidentiary reference for associating subsequent packet captures and analytical outputs with the specific laboratory exercise.

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ cat > reports/case_information.txt <<'EOF'
Case/Lab Identifier: SBT-DF203-Lab1-Kafayat-Animashawun
Trainee Name: Kafayat Animashawun
Date: 08 September 2026
Environment: Kali Linux VM
Purpose: Controlled plaintext HTTP traffic analysis
EOF

┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ cat reports/case_information.txt
Case/Lab Identifier: SBT-DF203-Lab1-Kafayat-Animashawun
Trainee Name: Kafayat Animashawun
Date: 08 September 2026
Environment: Kali Linux VM
Purpose: Controlled plaintext HTTP traffic analysis
```

Starting the Apache Web Server

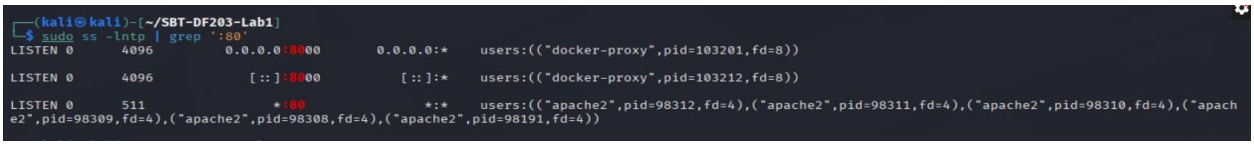
The Apache HTTP Server was enabled and started within the Kali Linux virtual machine using the system service manager. The service status output confirmed that apache2.service was loaded and in an active running state, with the Apache processes successfully launched. The service was also configured to start automatically through the system's service management mechanism. This step established the local web server required for generating controlled HTTP traffic. The Apache service therefore served as the server-side component of the communication session that was subsequently captured and examined using Wireshark and TShark.

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ sudo systemctl enable --now apache2
[sudo] password for kali:
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' → '/usr/lib/systemd/system/apache2.service'.

┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
     Active: active (running) since Tue 2026-09-08 14:18:53 EDT; 10s ago
 Invocation: 15f8c96329234f26a0a95aed68e17826
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 98191 (apache2)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 6 (limit: 8113)
     Memory: 45.7M (peak: 46M)
        CPU: 570ms
     CGroup: /system.slice/apache2.service
             ├─98191 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─98308 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─98309 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─98310 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─98311 /usr/sbin/apache2 -k start -DFOREGROUND
             └─98312 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 08 14:18:52 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 08 14:18:53 kali apachectl[98191]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.…his message
Sep 08 14:18:53 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```

Apache Listening on TCP Port 80

The local listening ports were examined using the ss command to confirm that the Apache service was accepting connections on the expected HTTP port. The output showed Apache listening on *:80, confirming that TCP port 80 was actively associated with the Apache processes. The output also showed an unrelated Docker proxy listening on port 8000; however, this did not interfere with the HTTP laboratory service because the

Apache server was bound to port 80. This verification established the server endpoint that would be contacted by the HTTP client during the controlled traffic-generation phase.



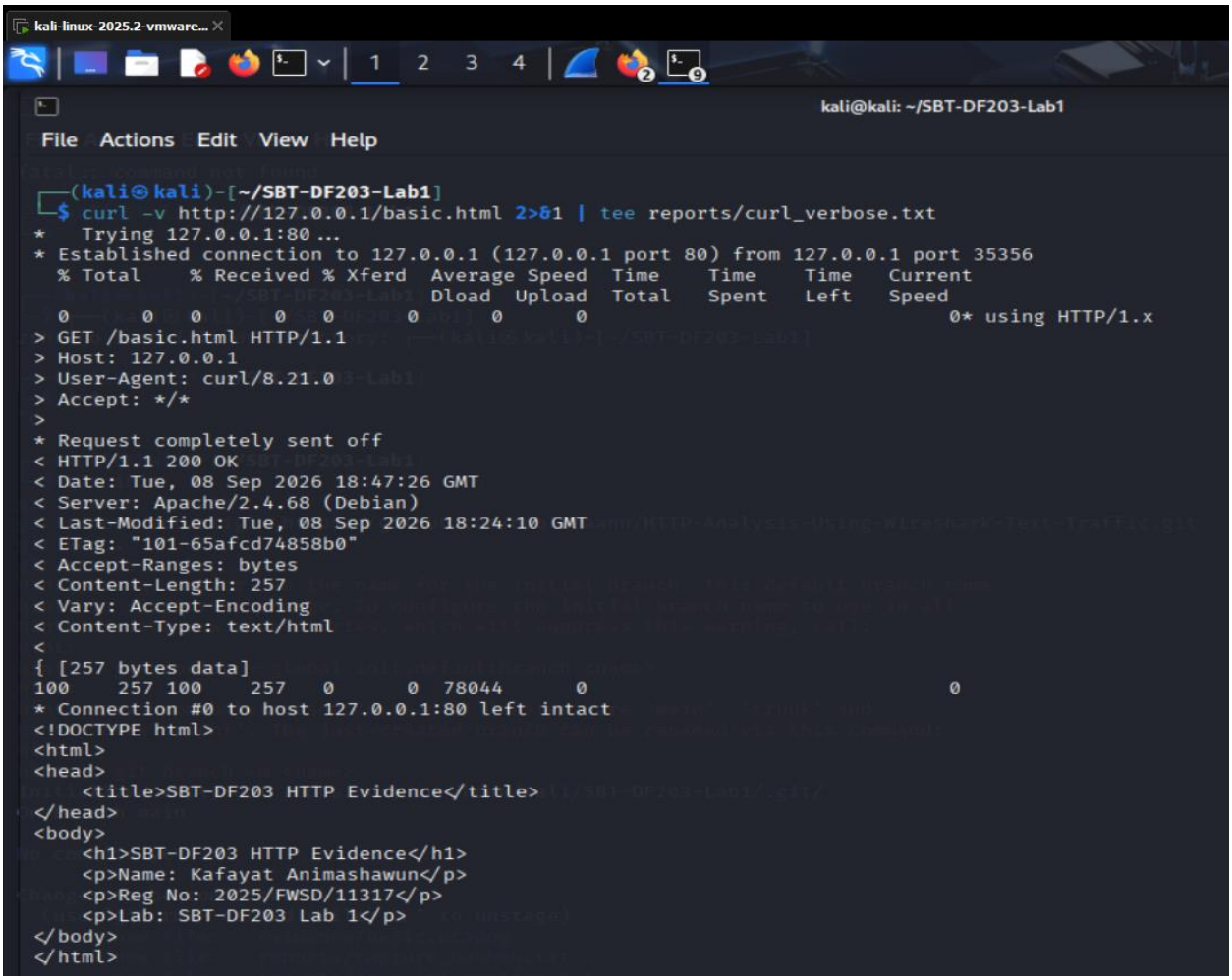
Creation of the Personalized HTML Evidence Page

A dedicated HTML page named basic.html was created in Apache's web document directory. The page was specifically prepared for the laboratory exercise and contained the title SBT-DF203 HTTP Evidence, the trainee's name, registration number, and the identification of the practical as SBT-DF203 Lab 1. The inclusion of these identifying details ensured that the HTTP response generated during the exercise could be directly associated with the student's practical work. The page also provided a controlled and predictable application-layer payload for subsequent HTTP packet analysis and TCP stream reconstruction.

Initial HTTP Request to the Local Web Server

The locally hosted webpage was tested using curl by requesting http://127.0.0.1/basic.html. The command returned the complete HTML document containing the SBT-DF203 evidence heading, the trainee's name, registration number and laboratory identification. The successful retrieval demonstrated that the Apache server was correctly configured and that the requested resource was available through the local HTTP service. This preliminary connectivity test was performed before packet

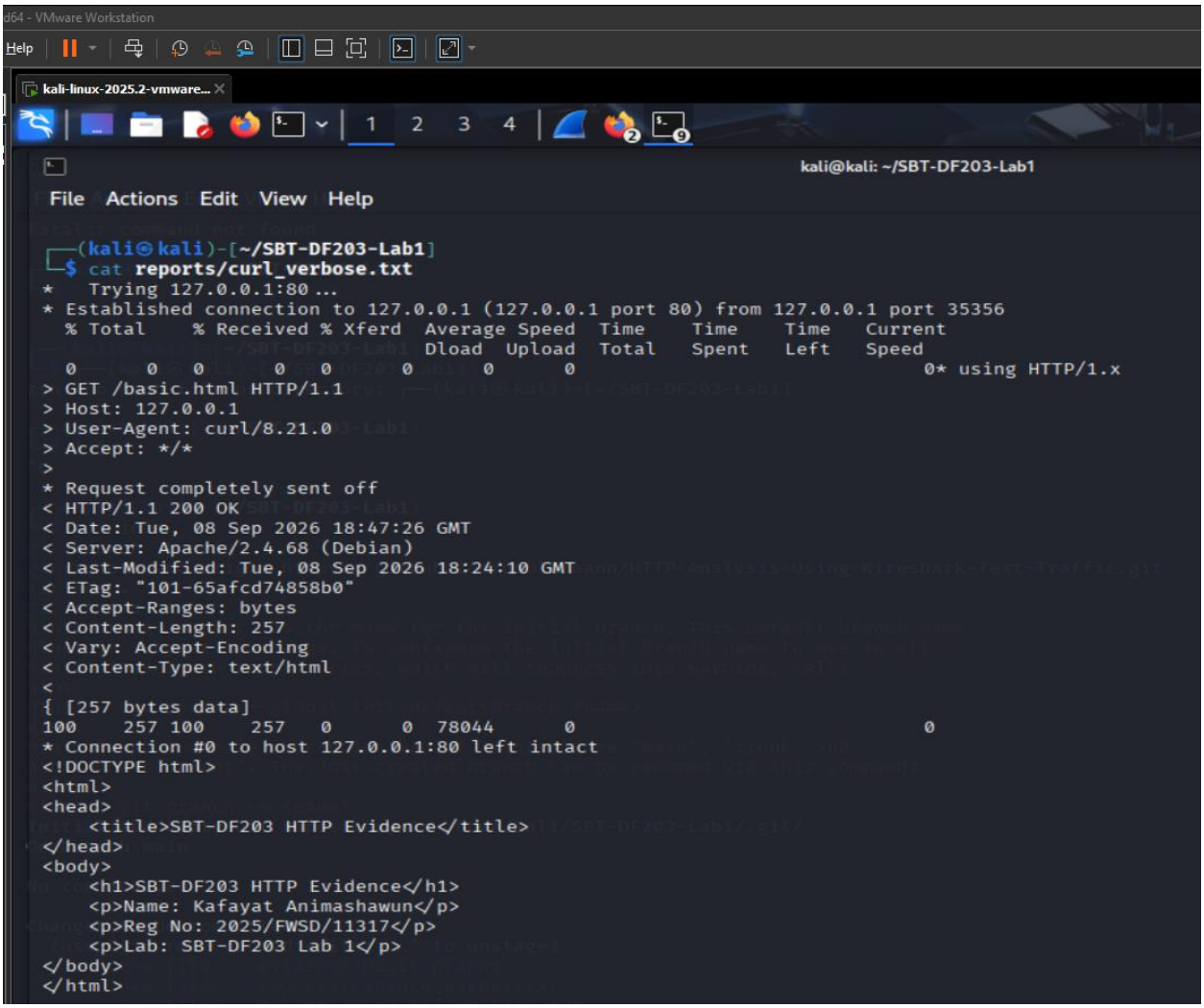
capture to confirm that the application-layer communication could be generated successfully and that the expected HTML content would be available for later reconstruction from the captured traffic.



Verbose HTTP Request and Response

A verbose curl request was subsequently executed to obtain detailed information about the HTTP transaction. The output shows that the client connected from the loopback address 127.0.0.1 using ephemeral source port 35356 to the Apache server at 127.0.0.1 on destination port 80. The client issued a GET /basic.html HTTP/1.1 request and supplied the Host: 127.0.0.1, User-Agent: curl/8.21.0, and Accept: */* headers. The server responded with HTTP/1.1 200 OK, indicating successful retrieval of the requested resource. The response identified the server as Apache/2.4.68 (Debian), reported a content type of text/html, and specified a content length of 257 bytes. The response also

contained the server timestamp of 08 September 2026 at 18:47:26 GMT. This transaction provided an application-layer reference against which the later packet-level HTTP analysis could be compared.

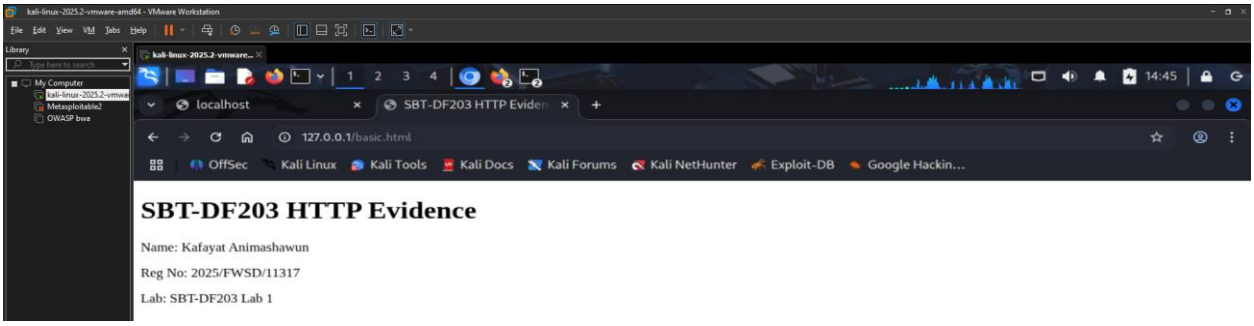


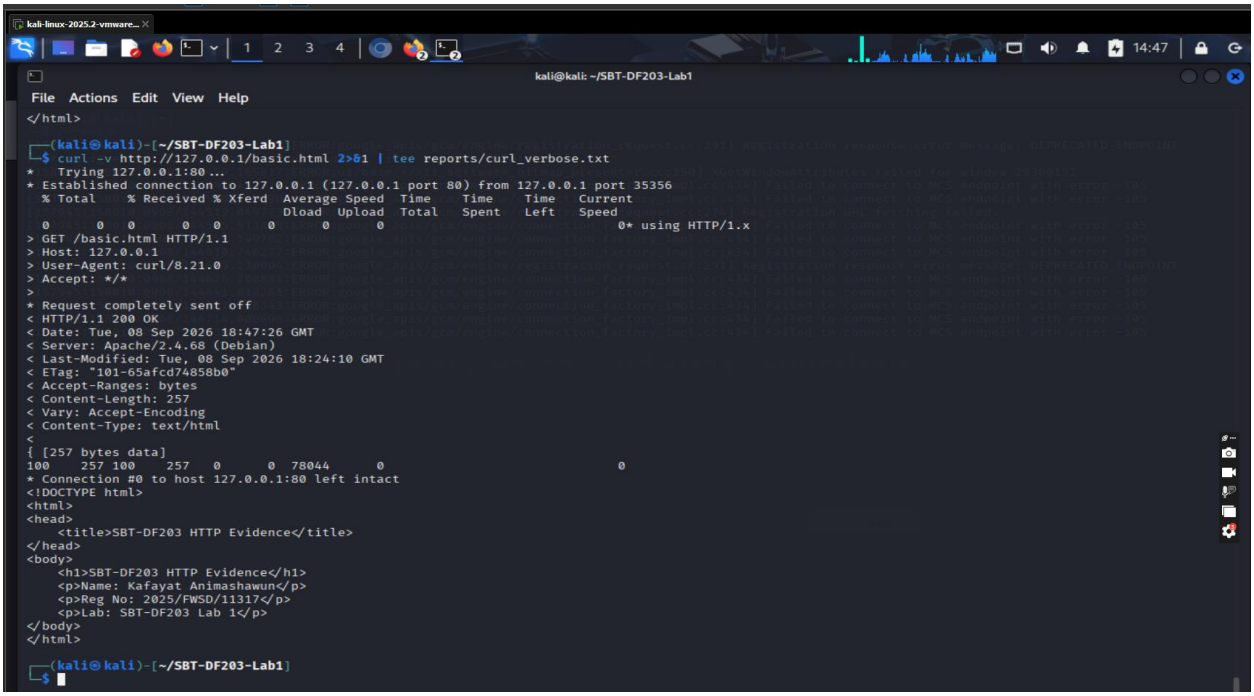
Verification of the HTTP Service After Traffic Generation

The Apache listening state was checked again after generating the HTTP request. The ss output continued to show Apache associated with TCP port 80, confirming that the web service remained available following the HTTP transaction. This verification demonstrated continuity of the local server environment and confirmed that the HTTP service had not terminated or changed its listening configuration during the practical exercise.

Browser-Based Verification

The Chromium browser was launched within the Kali Linux virtual machine to provide an additional method of accessing the locally hosted webpage. The terminal output contains Chromium startup messages and network-related warnings generated by the browser itself. These messages are not part of the HTTP evidence and do not indicate a failure of the Apache service. The browser activity was used to support visual verification of the locally hosted webpage and provided an additional means of confirming that the application-layer content created for the laboratory could be accessed through the local environment.

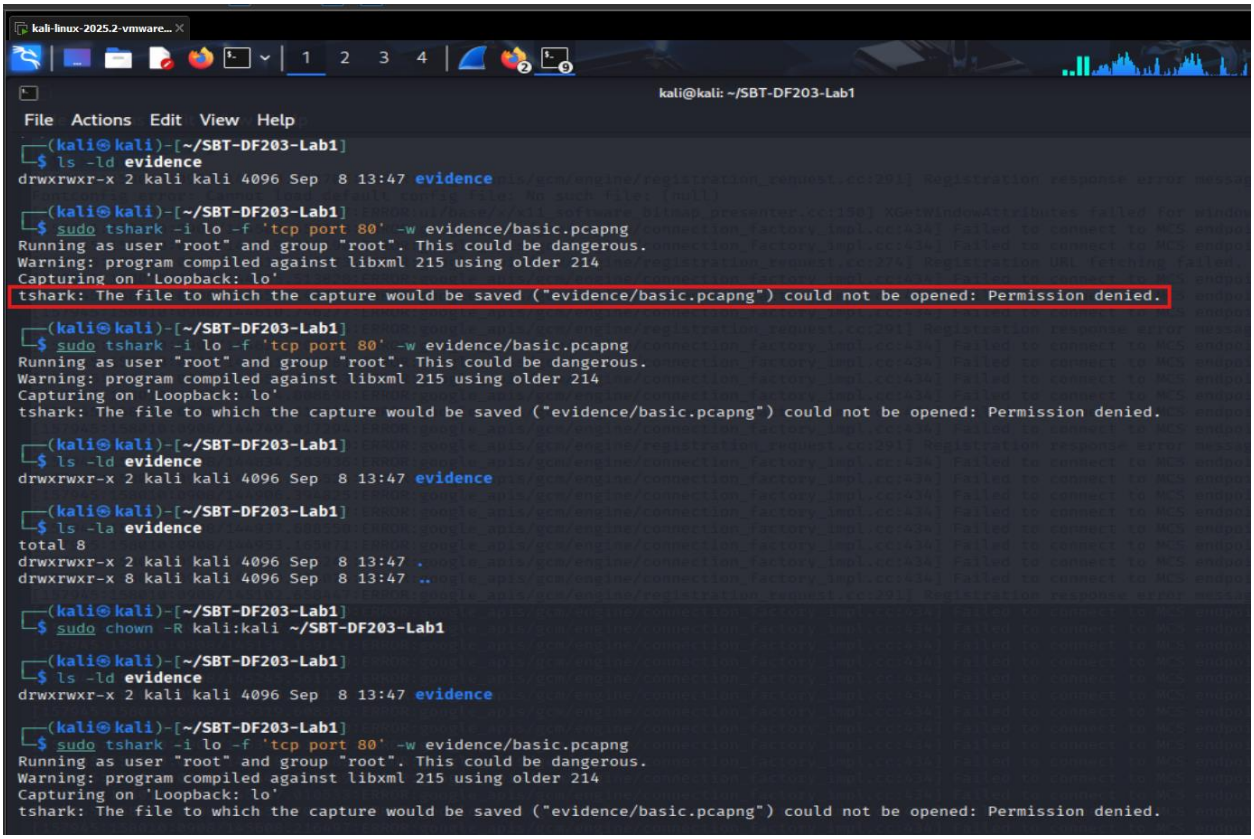




Initial TShark Capture Attempt and Permission Issue

An initial attempt was made to capture HTTP traffic from the loopback interface using TShark with the capture filter tcp port 80. Although TShark successfully identified the

loopback interface and began the capture process, it was unable to create the specified output file in the evidence directory and returned a Permission denied error. This demonstrated that packet acquisition had not yet successfully produced the intended evidence file. The issue was therefore investigated before proceeding with the final capture rather than treating the unsuccessful attempt as valid evidence.



Investigation of the Capture Permission Problem

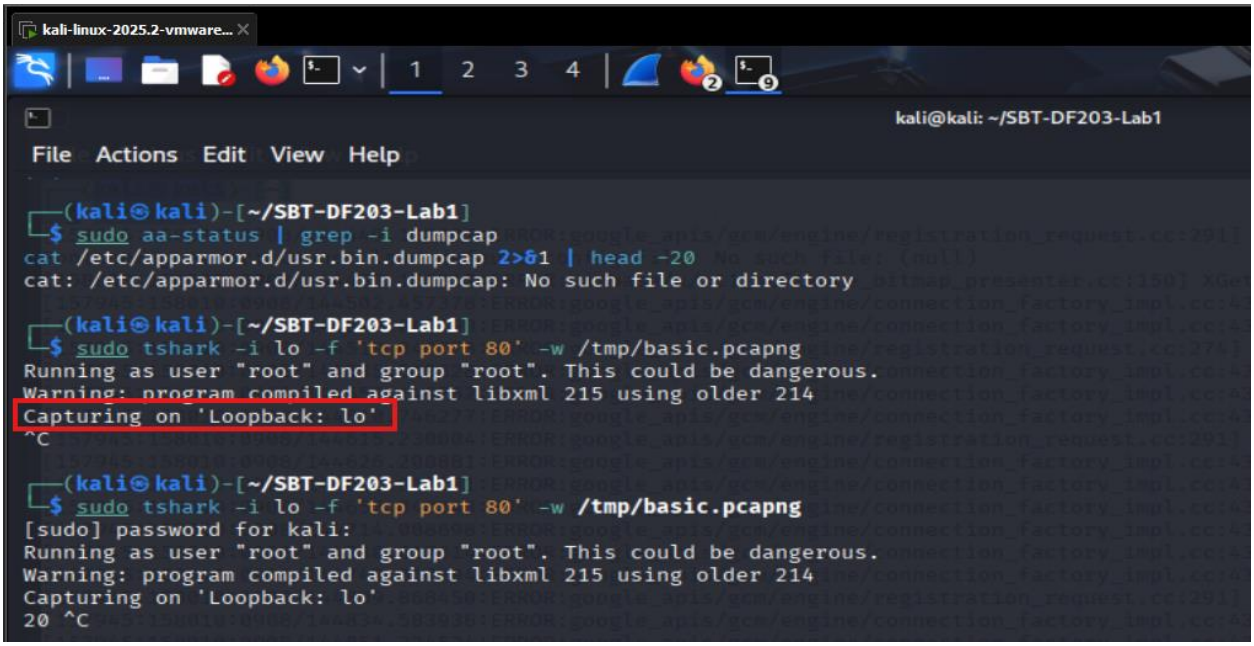
Following the failed TShark capture attempt, the permissions and filesystem characteristics of the evidence directory were investigated. The checks confirmed that the directory existed, was owned by the kali user and group, and had read/write/execute permissions for the owner and group. The underlying filesystem was identified as an ext4 filesystem mounted in read-write mode. Additional checks were performed using lsattr, mount information, user identity information and ACL inspection. These checks were conducted to determine whether filesystem permissions, mount restrictions or extended attributes were preventing TShark from creating the capture file. This troubleshooting

stage demonstrated a methodical approach to resolving an evidence-acquisition problem before continuing with packet collection.

The investigation also included an examination of possible security-policy restrictions affecting the TShark capture process. An attempt was made to determine whether AppArmor had a profile associated with dumpcap, the packet-capture component used by Wireshark and TShark. The expected AppArmor profile file was not present at the checked location. Although the exact root cause of the file-creation error is not established by the terminal evidence provided, these diagnostic commands demonstrate that the capture failure was investigated systematically rather than ignored. The unsuccessful attempts should be treated as troubleshooting activity rather than as part of the final packet evidence.

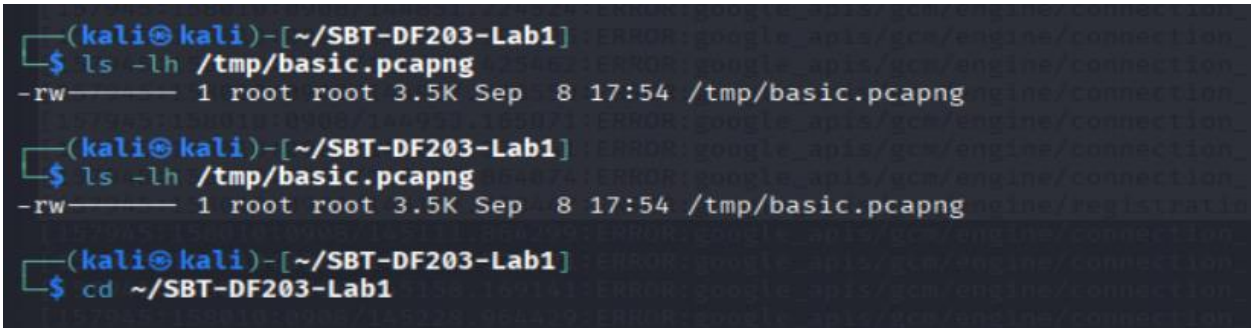
Successful TShark Packet Capture

After the earlier attempts to save the packet capture directly into the laboratory evidence directory resulted in a permission error, the capture destination was changed to the system temporary directory. TShark was executed against the loopback interface lo using the capture filter tcp port 80, restricting acquisition to TCP traffic associated with the HTTP service. The capture was allowed to run while the controlled HTTP transaction was generated and was then stopped using Ctrl+C. The successful capture produced the file /tmp/basic.pcapng, establishing a valid packet capture containing the HTTP communication between the local client and Apache web server. This approach ensured that packet acquisition could proceed without altering the traffic-generation methodology.



Verification of the Captured PCAPNG File

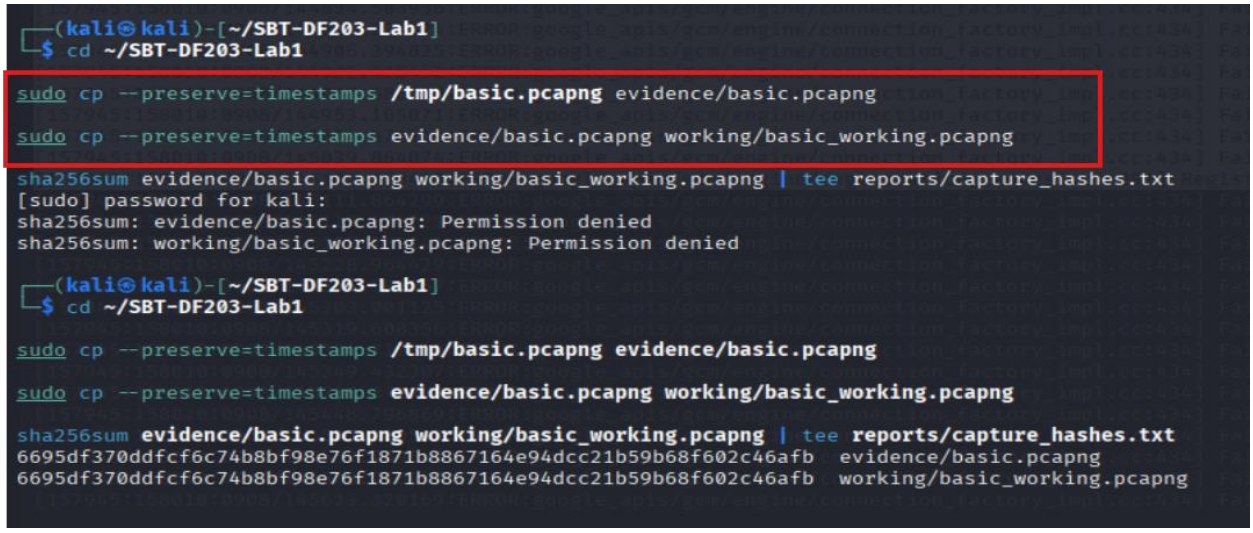
The resulting packet capture was verified using ls -lh /tmp/basic.pcapng. The output confirmed that the capture file existed and had a size of approximately 3.5 KB. The file was owned by the root user and group, consistent with the use of sudo during TShark execution. The presence and size of the file provided an initial confirmation that packet data had been successfully written to disk and that the capture could proceed to the evidence-preservation stage.



Preservation of the Original Evidence and Creation of a Working Copy

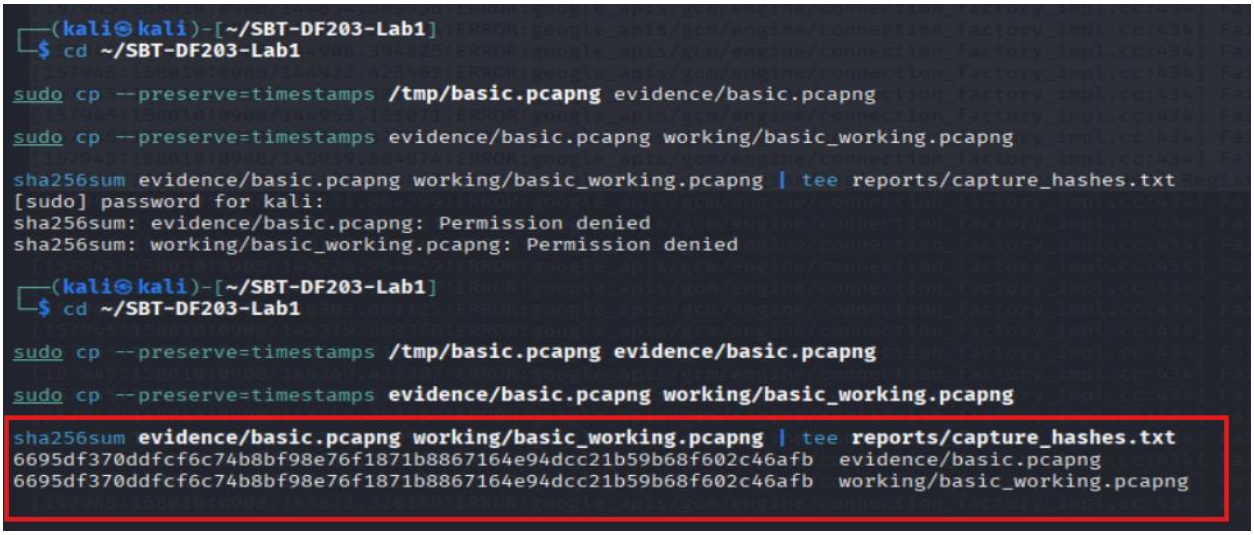
The successfully acquired packet capture was copied from /tmp/basic.pcapng into the laboratory evidence directory using cp --preserve=timestamps. A second copy was then created in the working directory as basic_working.pcapng. This established a separation between the original evidence copy and the working copy used for analysis. Preserving

timestamps during the copy operation also helped retain the original file metadata associated with the captured evidence. This evidence-handling process supports the forensic principle of preserving an original acquisition while conducting analysis on a separate working copy.



SHA-256 Hash Verification

SHA-256 cryptographic hashes were calculated for both evidence/basic.pcapng and working/basic_working.pcapng. The resulting hash value for both files was identical: 6695df370ddfcf6c74b8bf98e76f1871b8867164e94dcc21b59b68f602c46afb. The matching hashes demonstrate that the working copy was byte-for-byte consistent with the evidence copy at the time of verification. The hash values were also recorded in reports/capture_hashes.txt, providing a documented integrity reference for the packet capture. This step is particularly important in a digital-forensics investigation because it provides a reproducible mechanism for demonstrating that the analyzed working copy corresponds to the preserved evidence.



Complete TCP Packet Timeline

The working packet capture was examined using TShark to extract the frame number, timestamp, source and destination addresses, TCP ports, TCP flags, sequence numbers, acknowledgement numbers and TCP payload lengths. The capture contains two complete HTTP/TCP exchanges. The first connection uses client source port 43594 and server port 80, while the second uses client source port 53484 and server port 80. Both communications occur between 127.0.0.1 and 127.0.0.1, confirming that the traffic was generated locally through the loopback interface. For the first session, frame 1 contains the initial SYN, frame 2 contains SYN-ACK, and frame 3 contains the final ACK, establishing the TCP three-way handshake. Frame 4 carries the HTTP request with 83 bytes of TCP payload, followed by an acknowledgement in frame 5. Frame 6 contains the 509-byte server response, which is acknowledged by frame 7. Frames 8 and 9 contain FIN/ACK packets for connection termination, followed by the final acknowledgement in frame 10. The second session follows the same sequence, beginning at frame 11 and terminating at frame 20. This packet-level sequence provides the chronological foundation for reconstructing the HTTP communication.

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ tshark -r working/basic_working.pcapng -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e tcp.flags -e tcp.seq \
-e tcp.ack -e tcp.len
Warning: program compiled against libxml 215 using older 214
1       2026-09-08T17:47:02.868998488-0400      127.0.0.1       43594   127.0.0.1       80      0x0002  0       0       0
2       2026-09-08T17:47:02.869066447-0400      127.0.0.1       80      127.0.0.1       43594   0x0012  0       1       0
3       2026-09-08T17:47:02.869084656-0400      127.0.0.1       43594   127.0.0.1       80      0x0010  1       1       0
4       2026-09-08T17:47:02.869569109-0400      127.0.0.1       43594   127.0.0.1       80      0x0018  1       1       83
5       2026-09-08T17:47:02.869991445-0400      127.0.0.1       80      127.0.0.1       43594   0x0010  1       84      0
6       2026-09-08T17:47:02.929539113-0400      127.0.0.1       80      127.0.0.1       43594   0x0018  1       84      509
7       2026-09-08T17:47:02.929571637-0400      127.0.0.1       43594   127.0.0.1       80      0x0010  84      510     0
8       2026-09-08T17:47:02.929926311-0400      127.0.0.1       43594   127.0.0.1       80      0x0011  84      510     0
9       2026-09-08T17:47:02.933988683-0400      127.0.0.1       80      127.0.0.1       43594   0x0011  510     85      0
10      2026-09-08T17:47:02.934017109-0400      127.0.0.1       43594   127.0.0.1       80      0x0010  85      511     0
11      2026-09-08T17:48:56.825026000-0400      127.0.0.1       53484   127.0.0.1       80      0x0002  0       0       0
12      2026-09-08T17:48:56.825478971-0400      127.0.0.1       80      127.0.0.1       53484   0x0012  0       1       0
13      2026-09-08T17:48:56.825501950-0400      127.0.0.1       53484   127.0.0.1       80      0x0010  1       1       0
14      2026-09-08T17:48:56.826892644-0400      127.0.0.1       53484   127.0.0.1       80      0x0018  1       1       83
15      2026-09-08T17:48:56.827410257-0400      127.0.0.1       80      127.0.0.1       53484   0x0010  1       84      0
16      2026-09-08T17:48:56.834637942-0400      127.0.0.1       80      127.0.0.1       53484   0x0018  1       84      509
17      2026-09-08T17:48:56.834712117-0400      127.0.0.1       53484   127.0.0.1       80      0x0010  84      510     0
18      2026-09-08T17:48:56.835635839-0400      127.0.0.1       53484   127.0.0.1       80      0x0011  84      510     0
19      2026-09-08T17:48:56.836398673-0400      127.0.0.1       80      127.0.0.1       53484   0x0011  510     85      0
20      2026-09-08T17:48:56.836429222-0400      127.0.0.1       53484   127.0.0.1       80      0x0010  85      511     0
```

TCP Three-Way Handshake

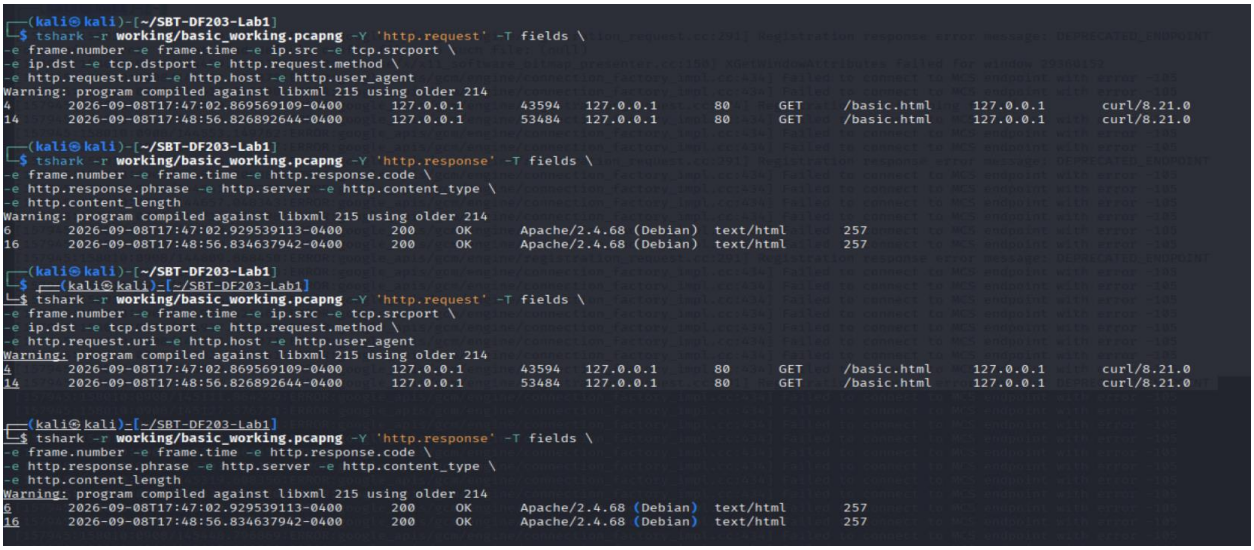
The packet timeline demonstrates a complete TCP three-way handshake for the first HTTP session. Frame 1 originates from 127.0.0.1:43594 and is directed to 127.0.0.1:80 with the SYN flag set. The client begins with sequence number 0. Frame 2 is returned from 127.0.0.1:80 to the client and contains both SYN and ACK flags, with sequence number 0 and acknowledgement number 1. The acknowledgement of 1 indicates that the server has received the client's SYN and is expecting the next sequence number, with the SYN consuming one sequence-number position. Frame 3 is the client's final ACK, with sequence number 1 and acknowledgement number 1. At this point, the TCP connection is established and the endpoints can exchange application data. The same three-way handshake pattern is repeated for the second session beginning with frames 11–13, confirming that both HTTP transactions were carried over successfully established TCP connections.

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e http.request.method \
-e http.request.uri -e http.host -e http.user_agent
Warning: program compiled against libxml 215 using older 214
4       2026-09-08T17:47:02.869569109-0400      127.0.0.1       43594   127.0.0.1       80      GET     /basic.html     127.0.0.1       curl/8.21.0
14      2026-09-08T17:48:56.826892644-0400      127.0.0.1       53484   127.0.0.1       80      GET     /basic.html     127.0.0.1       curl/8.21.0
```

HTTP GET Request Analysis

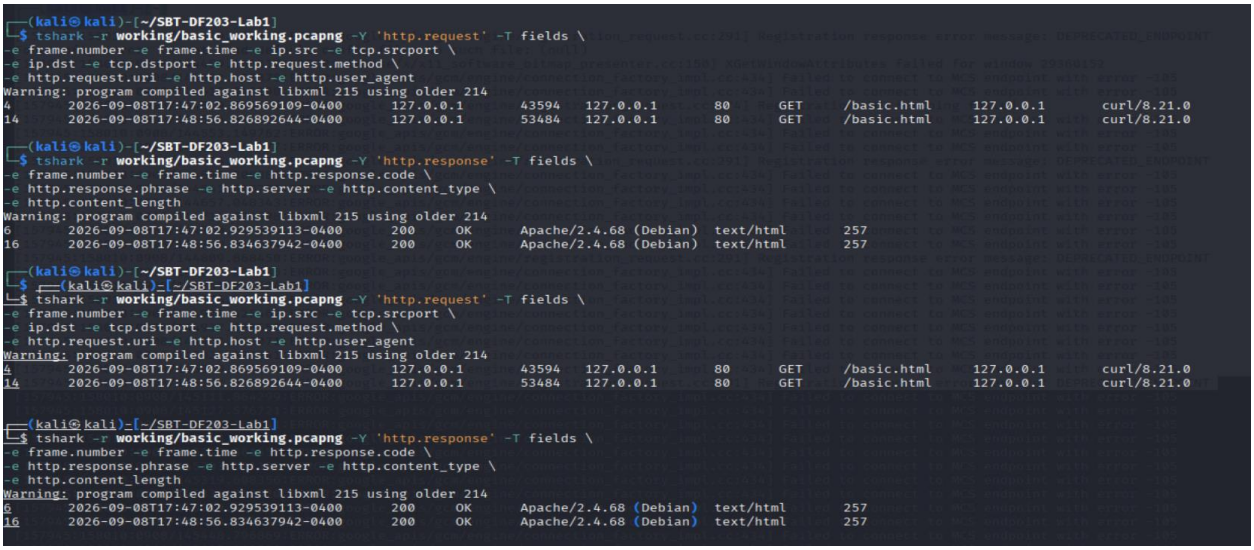
TShark filtering for http.request identified two HTTP GET requests in the captured traffic. The first request occurs in frame 4 at timestamp 2026-09-08T17:47:02.869569109-0400, originating from 127.0.0.1:43594 and destined for 127.0.0.1:80. The request method is GET, and the requested resource is /basic.html. The HTTP Host header identifies

127.0.0.1, while the User-Agent identifies the client as curl/8.21.0. A second, equivalent GET request occurs in frame 14 at 17:48:56.826892644-0400, using source port 53484. The presence of the two requests confirms that the captured traffic contains repeated controlled retrievals of the same locally hosted resource. The packet evidence therefore directly demonstrates the HTTP request stage of the client-server communication.



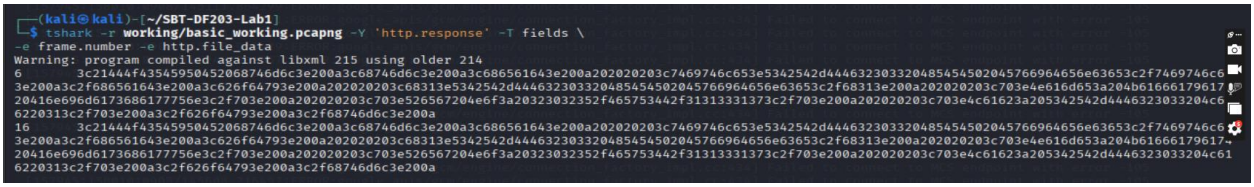
HTTP 200 OK Response

The corresponding HTTP responses were identified using the http.response display filter. Frame 6 contains the response to the first request and reports HTTP status code 200 with the reason phrase OK. The server is identified as Apache/2.4.68 (Debian), while the content type is text/html and the content length is 257 bytes. The same response characteristics are present in frame 16 for the second HTTP transaction. The HTTP 200 status confirms that Apache successfully located and returned the requested /basic.html resource. These fields provide direct application-layer evidence linking the TCP communication to the successful retrieval of the laboratory webpage.



HTTP Request and Response Correlation

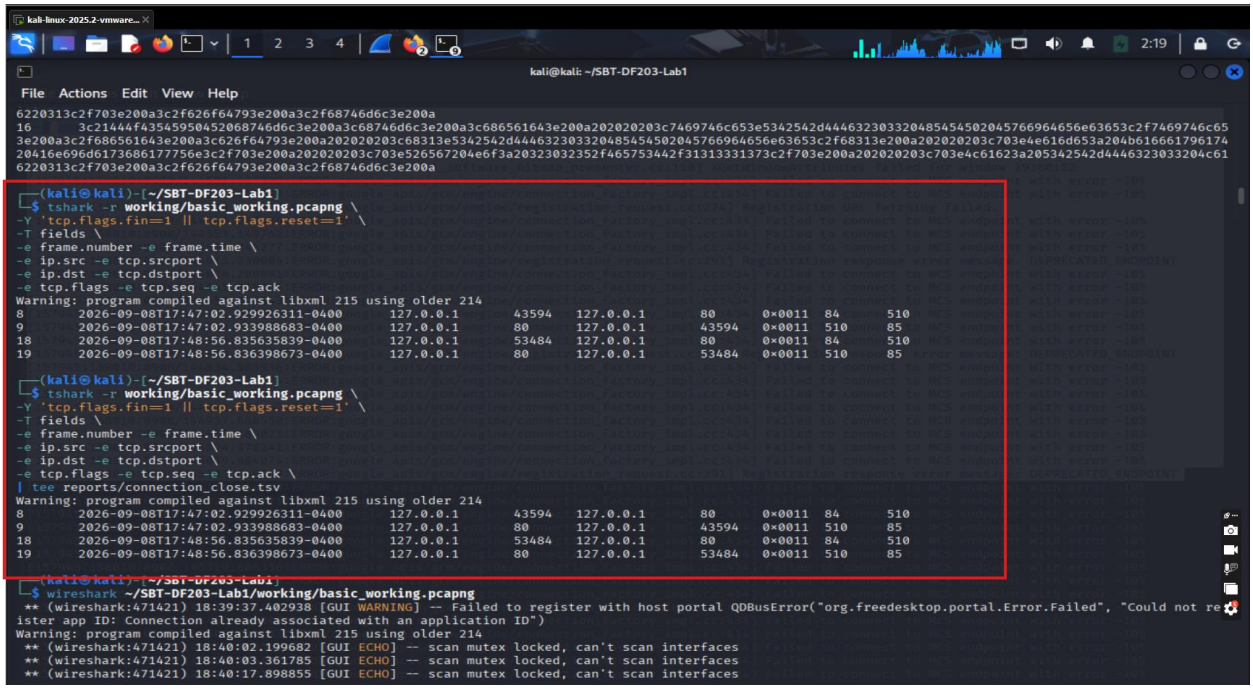
The combined HTTP analysis demonstrates a clear request-response relationship within each TCP session. In the first session, frame 4 contains the client's GET /basic.html request, while frame 6 contains the corresponding HTTP/1.1 200 OK response from Apache. The second session follows the same pattern with the GET request in frame 14 and the successful response in frame 16. In both cases, the server identifies itself as Apache/2.4.68 (Debian), returns HTML content of 257 bytes, and communicates over TCP port 80. The repeated pattern confirms that the traffic was deterministic and generated from the controlled laboratory webpage rather than representing unrelated network activity.



Extraction of the HTML Payload

The HTTP response payload was extracted from frames 6 and 16 using the http.file_data field. The resulting hexadecimal data represents the HTML content transmitted by the Apache server. Decoding the hexadecimal representation reveals the document structure, including the SBT-DF203 HTTP Evidence title and heading, the trainee's name, the registration number 2025/FWSD/11317, and the laboratory identifier SBT-DF203 Lab

1. This demonstrates an important forensic characteristic of plaintext HTTP: application-layer content can be reconstructed directly from captured network traffic because the HTTP payload is not encrypted. The extracted payload therefore provides packet-level corroboration of the webpage content created earlier in the investigation.



TCP Connection Termination

The TCP connection closure was examined by filtering for packets containing either the FIN or RST flags. The results show a normal, orderly TCP termination for both captured HTTP sessions. For the first connection, frame 8 is sent from the client 127.0.0.1:43594 with FIN/ACK flags, sequence number 84 and acknowledgement number 510. Frame 9 is returned by the server with FIN/ACK, sequence number 510 and acknowledgement number 85. Frame 10 then provides the client's final ACK with sequence number 85 and acknowledgement number 511. The second connection exhibits the same termination pattern in frames 18–20. The absence of RST packets in the displayed termination results indicates that the captured sessions were closed through the normal TCP FIN/ACK process rather than being abruptly reset.

Second HTTP Session

The packet capture contains a second complete HTTP session beginning at frame 11. The client uses ephemeral port 53484 to connect to Apache on TCP port 80. Frames 11–13 establish the TCP connection through the SYN, SYN-ACK and ACK sequence. Frame 14 then carries the second GET /basic.html request with an 83-byte TCP payload, followed by the server acknowledgement in frame 15. Frame 16 contains the 509-byte HTTP response, which is acknowledged in frame 17. The session then closes normally through frames 18–20. The second complete exchange provides additional corroboration that the Apache service consistently returned the expected resource and that the packet capture contains more than one successful HTTP transaction.

Web Service Verification and HTTP Traffic Generation

The Apache2 web server was configured to host the controlled training webpage for the laboratory. The service status confirmed that apache2.service was active and running, having started at 14:18:53 EDT on 8 September 2026. The command ss -lntp | grep ':80' further confirmed that Apache2 was listening on TCP port 80. The training webpage was located at /var/www/html/basic.html and had a recorded file size of 257 bytes.

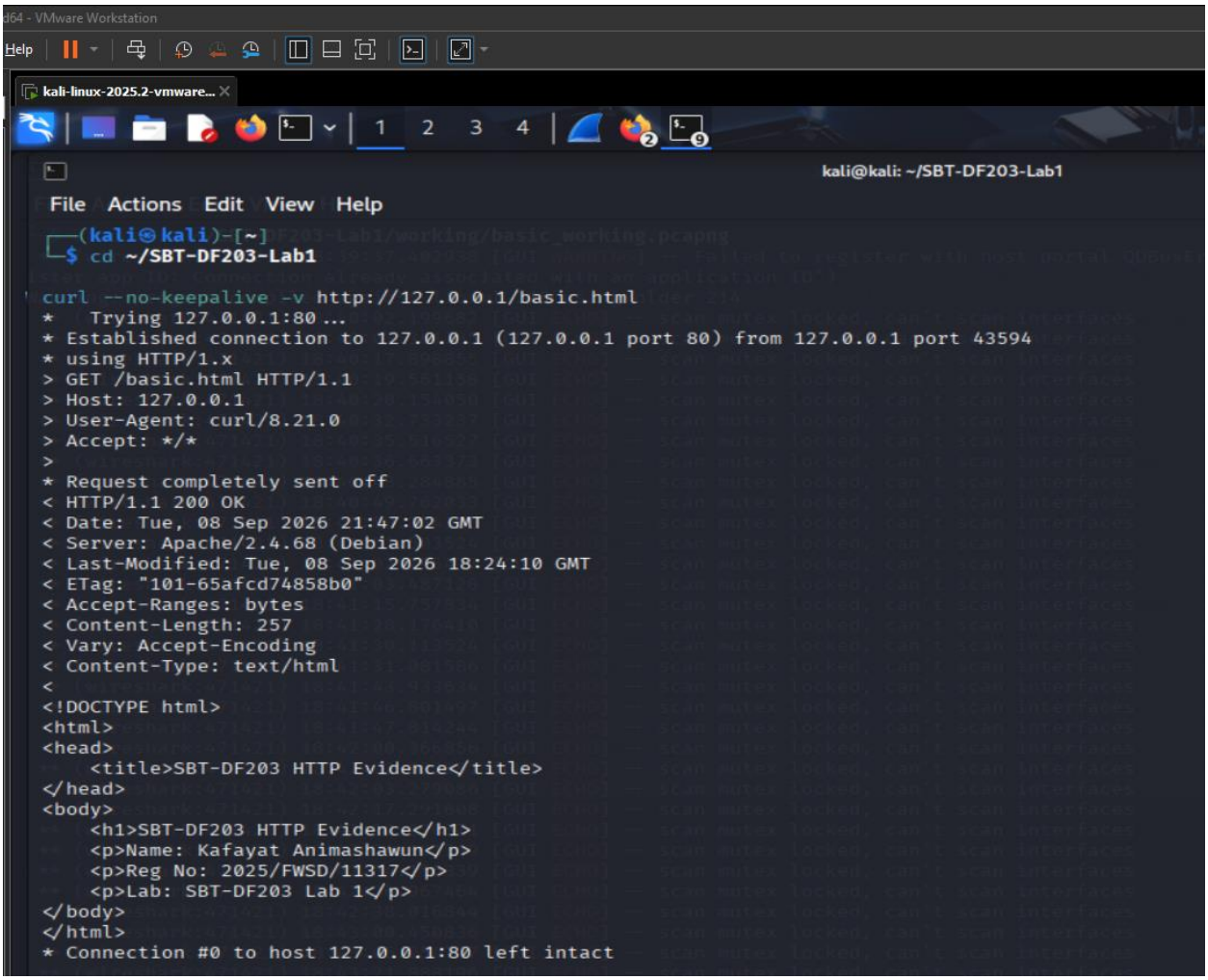
The webpage was accessed exclusively through the local loopback address 127.0.0.1, ensuring that the HTTP traffic remained within the authorised virtual laboratory environment. The resource requested was /basic.html, which contained the laboratory title, the trainee's name Kafayat Animashawun, registration number 2025/FWSD/11317, and the laboratory identifier SBT-DF203 Lab 1.

A controlled HTTP session was generated using the curl --no-keepalive -v command. The first observed connection was established from the loopback client 127.0.0.1:43594 to the Apache server at 127.0.0.1:80. The client issued a GET /basic.html HTTP/1.1 request containing the Host: 127.0.0.1, User-Agent: curl/8.21.0, and Accept: */* headers. The server successfully returned HTTP/1.1 200 OK, confirming successful retrieval of the requested resource.

The HTTP response identified the web server as Apache/2.4.68 (Debian) and specified Content-Type: text/html with a Content-Length of 257 bytes. Other observed response headers included Date, Last-Modified, ETag, Accept-Ranges, and Vary. The response

body contained the complete HTML training page, allowing the application-layer content to be observed directly as plaintext.

A second controlled request was subsequently generated and recorded using client port 53484. It produced the same HTTP request and response structure and returned the same 257-byte HTML resource. The second response was timestamped 21:48:56 GMT on 8 September 2026. These observations demonstrate that the local Apache service was functioning correctly and that repeatable plaintext HTTP traffic was successfully generated for packet capture and forensic analysis.



Forensic Significance of the HTTP Evidence

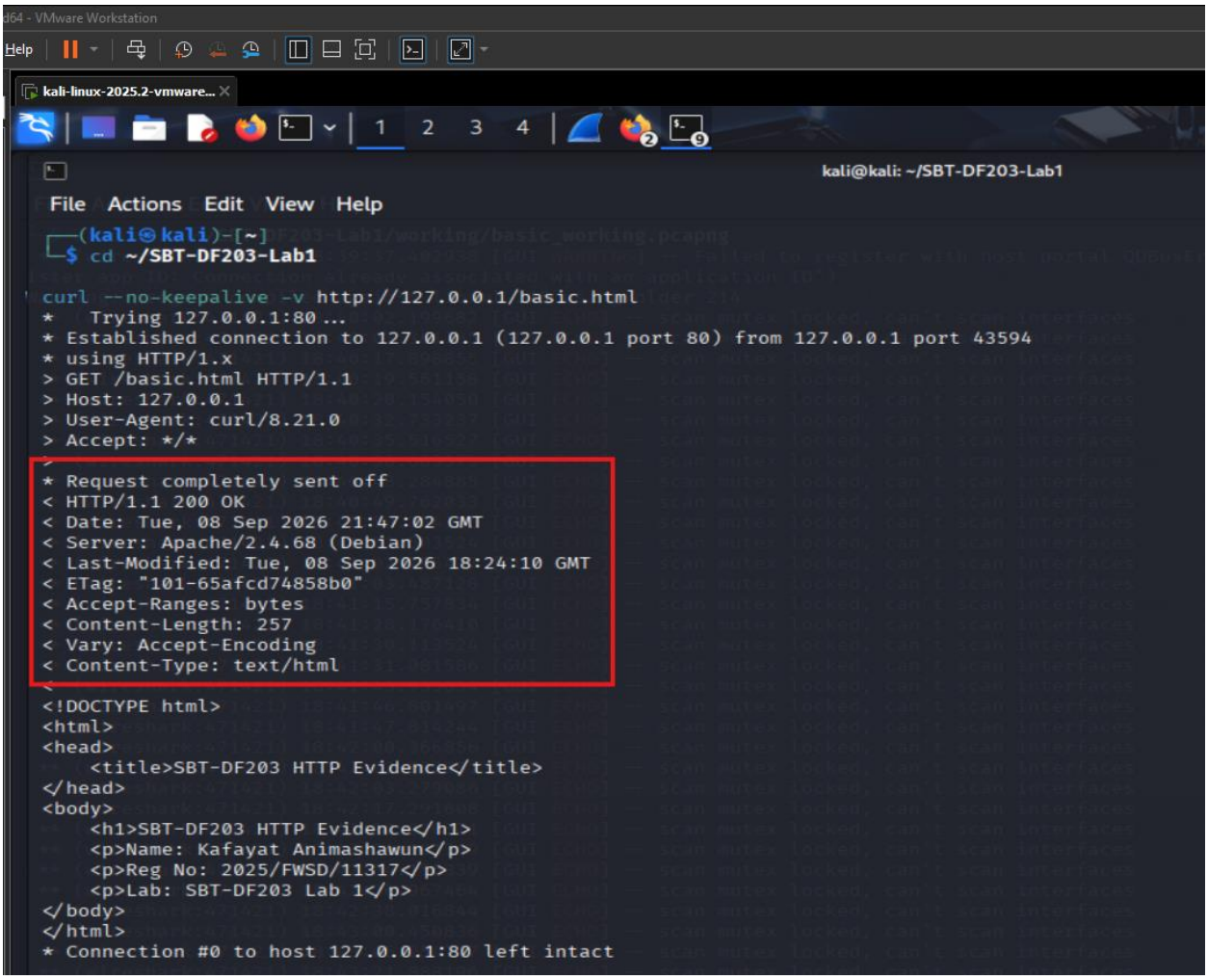
The captured HTTP exchange demonstrates the evidential value of plaintext application-layer traffic. The request exposes the HTTP method, requested resource, destination host

and client software through the request headers. In this session, the client requested /basic.html using the GET method and identified itself as curl/8.21.0. The corresponding server response provides additional evidence, including the HTTP status code, web-server software, content type and content length.

Because the exercise used unencrypted HTTP within a controlled environment, the content of the requested webpage was transmitted in plaintext and could therefore be reconstructed from the TCP conversation. The response body contained the laboratory identification information, providing a direct link between the generated application traffic and the intended forensic exercise.

At the transport layer, the connections demonstrate the use of TCP with the standard HTTP service port 80 and dynamically assigned client-side ephemeral ports. The first observed connection used source port 43594, while the subsequent connection used source port 53484. This combination of IP addresses and ports provides the information required to distinguish the client and server endpoints and associate packets with their respective TCP conversations.

The timestamps also provide temporal evidence for the reconstruction of the activity. The first HTTP response was recorded with a server Date value of 21:47:02 GMT, while the second response was recorded at 21:48:56 GMT. The corresponding TShark capture timestamps use the local EDT offset (-0400), which is equivalent to GMT/UTC after accounting for the four-hour difference. This demonstrates that the packet-capture timestamps and HTTP-layer timestamp information are temporally consistent.



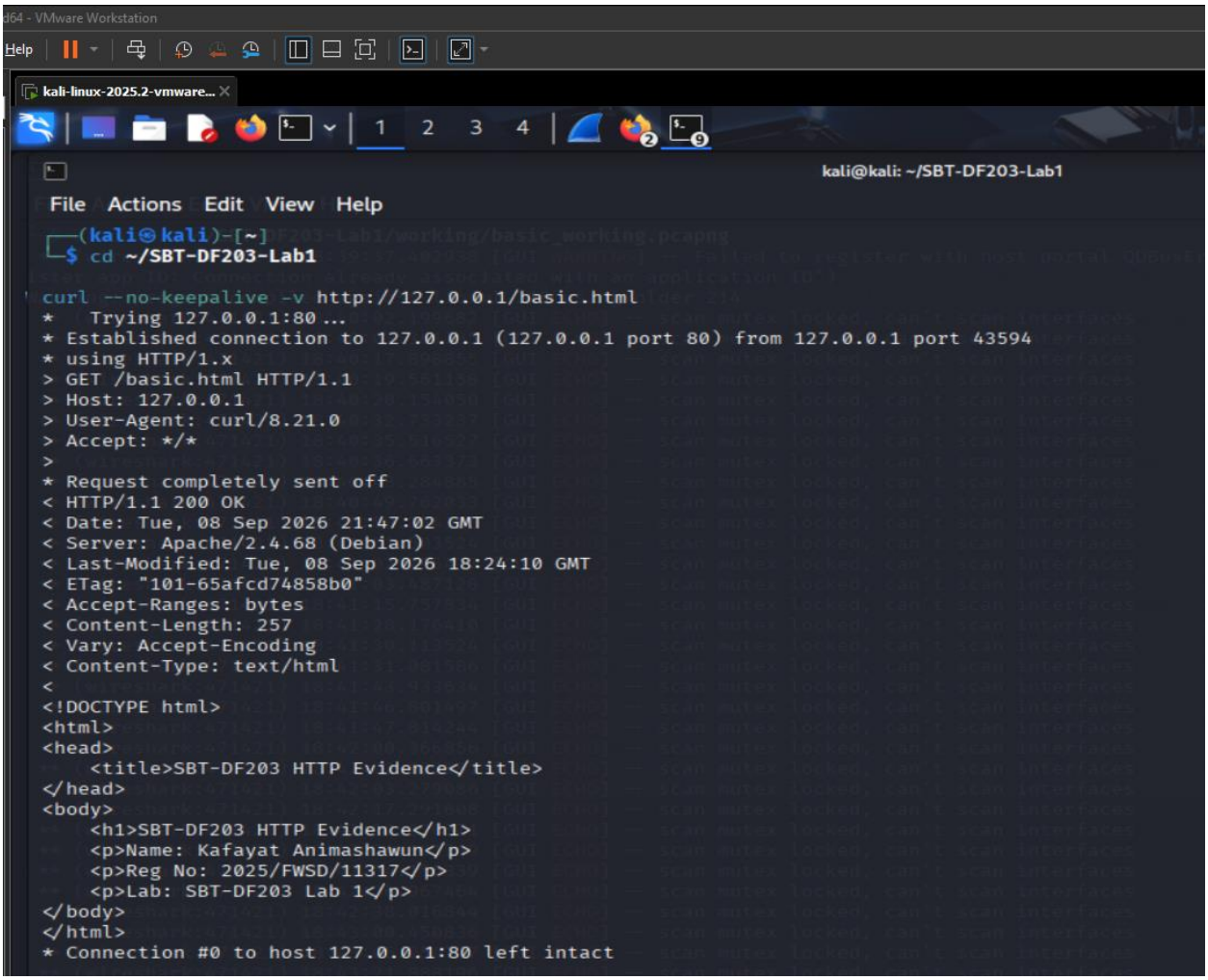
TCP Connection Closure

The capture was examined for TCP FIN and RST flags to establish how the HTTP connections terminated. The analysis identified FIN/ACK packets for both controlled sessions. For the first session, frame 8 was transmitted from 127.0.0.1:43594 to 127.0.0.1:80 with TCP flags 0x0011, representing FIN and ACK. The packet had sequence number 84 and acknowledgement number 510. Frame 9 followed in the reverse direction from 127.0.0.1:80 to 127.0.0.1:43594, also containing FIN and ACK flags, with sequence number 510 and acknowledgement number 85.

The same termination pattern was observed for the second session. Frame 18 was transmitted from 127.0.0.1:53484 to 127.0.0.1:80, with sequence number 84 and acknowledgement number 510. Frame 19 was transmitted in the reverse direction, from

127.0.0.1:80 to 127.0.0.1:53484, with sequence number 510 and acknowledgement number 85.

The absence of RST packets in the displayed termination results and the presence of FIN/ACK exchanges indicate an orderly TCP connection shutdown for the two observed sessions. The acknowledgement values also demonstrate the progression of TCP sequence space during termination: the acknowledgement of 85 reflects acknowledgement of the peer's FIN following sequence number 84, since a TCP FIN consumes one sequence number. The termination packets therefore provide evidence of the normal conclusion of the controlled HTTP sessions.



Packet Capture Overview and HTTP Session Identification

The Wireshark capture file basic_working.pcapng contained 20 packets associated with the controlled HTTP activity on the loopback interface. The packet list shows communication exclusively between 127.0.0.1 endpoints, with TCP port 43594 and later 53484 acting as client-side ephemeral ports and TCP port 80 serving as the Apache HTTP destination. The capture therefore provides packet-level evidence of the locally generated HTTP transactions between the curl client and the Apache web server.

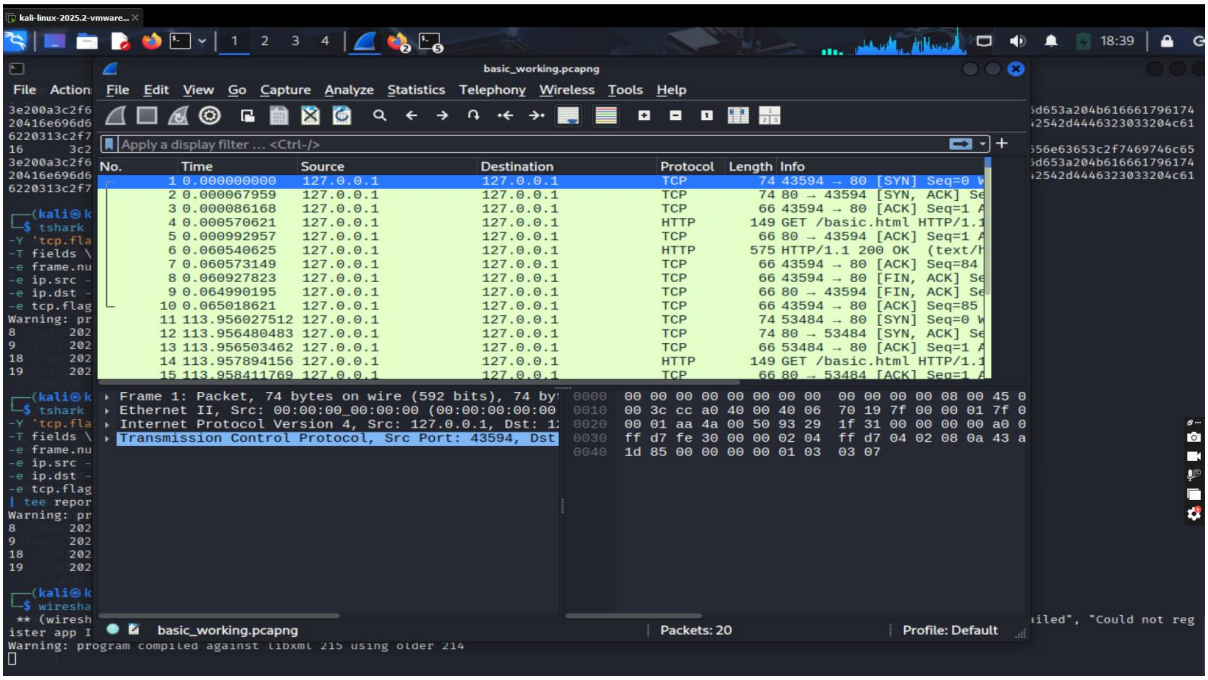
For the first observed HTTP session, the packet sequence includes the TCP connection establishment followed by the HTTP transaction. The packet list identifies a [SYN] packet from 127.0.0.1:43594 to 127.0.0.1:80, followed by the corresponding TCP response and acknowledgement, establishing the connection before application data was exchanged. The capture then records a GET /basic.html HTTP/1.1 request, followed by an HTTP/1.1 200 OK response from the server. Subsequent ACK packets confirm receipt of the transmitted data. The session concludes with FIN/ACK packets, providing evidence of an orderly TCP connection termination.

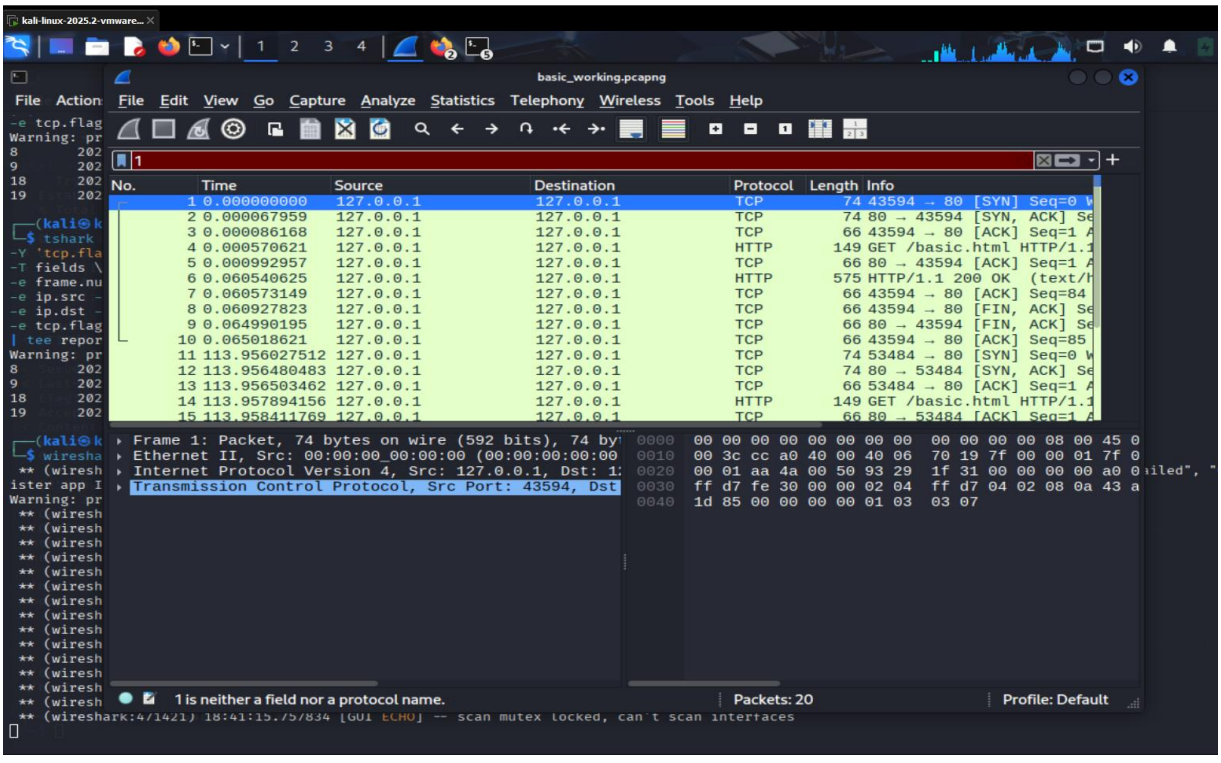
A second HTTP session is also visible in the same capture, using client port 53484. This session similarly contains the GET /basic.html HTTP/1.1 request and the corresponding HTTP/1.1 200 OK response. The presence of both 43594 and 53484 is significant because it demonstrates that the webpage retrieval was performed more than once, with each HTTP request associated with a distinct TCP client port. This also provides an opportunity to correlate the packet capture with the two controlled curl executions documented in the acquisition evidence.

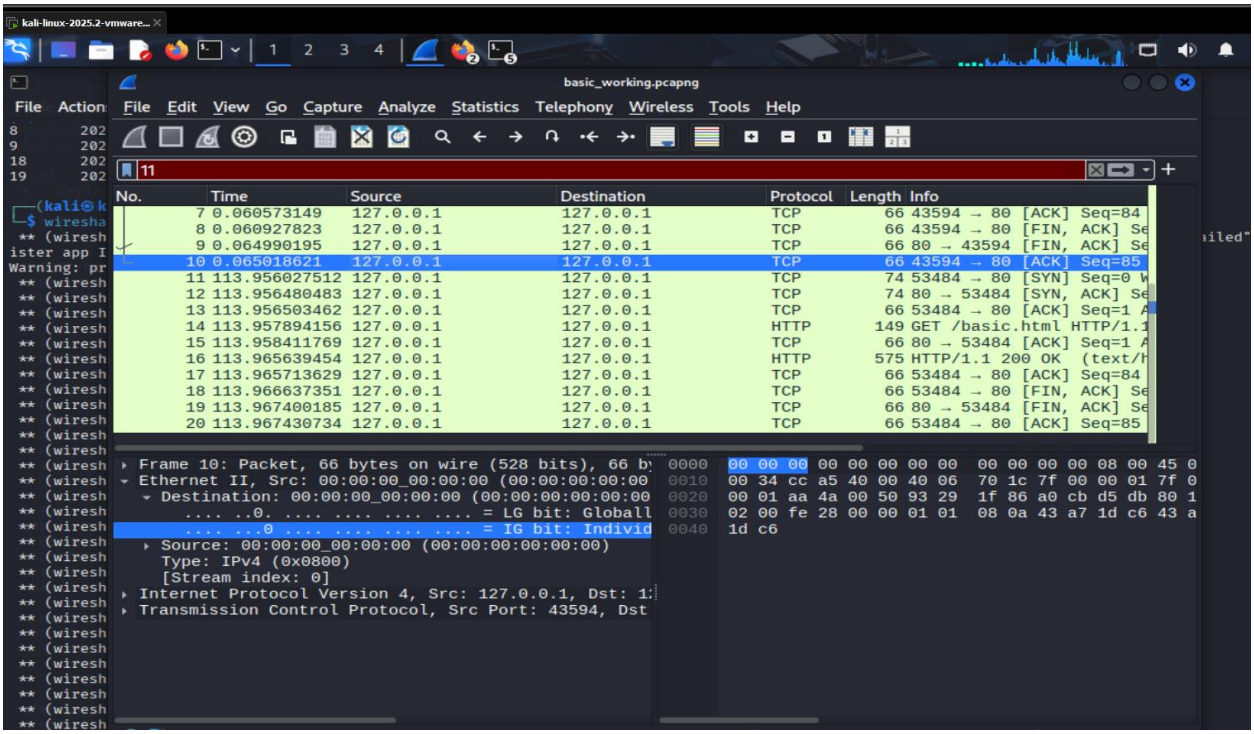
TCP and HTTP Packet Sequence

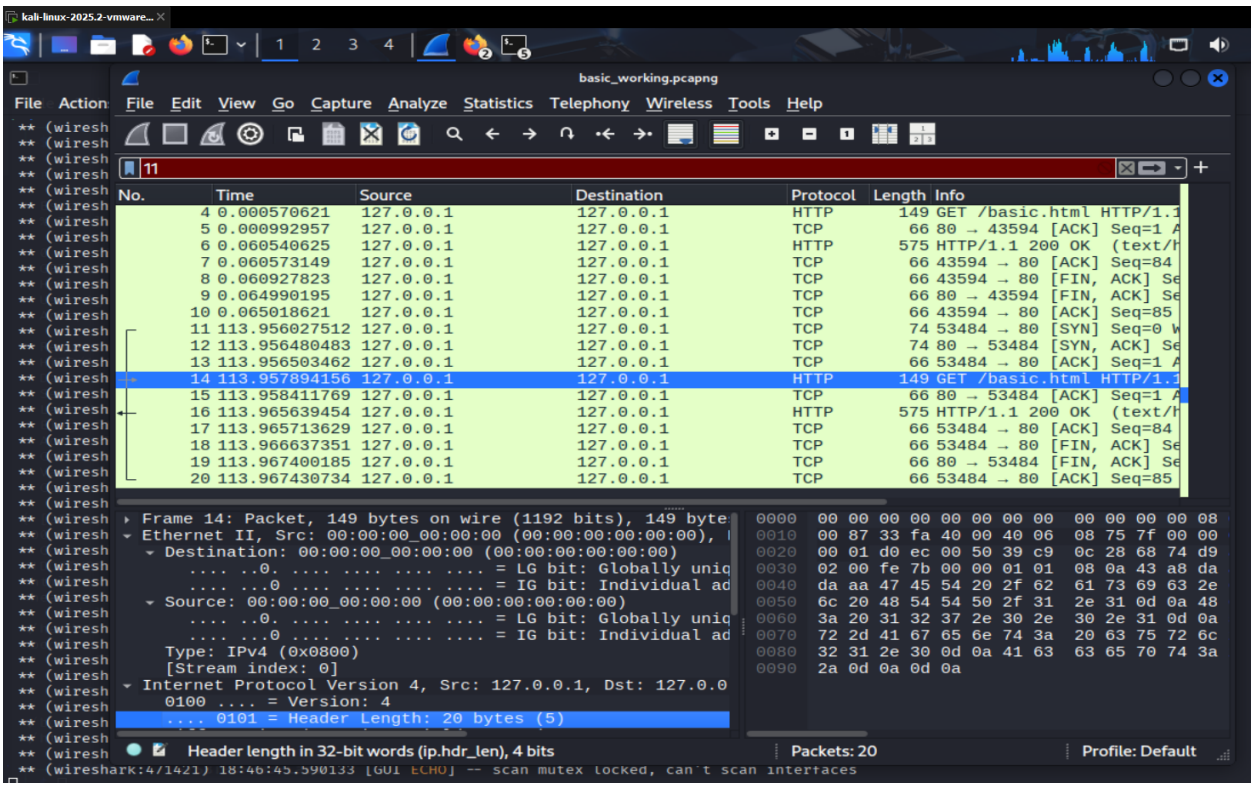
The visible packet sequence demonstrates the expected progression of the HTTP communication: TCP connection establishment → HTTP GET request → HTTP 200 response → TCP acknowledgements → connection termination. Wireshark identifies the HTTP request as GET /basic.html HTTP/1.1 and the response as HTTP/1.1 200 OK. Inspection of the selected packet also shows an IPv4 header with Protocol: TCP (6), source and destination address 127.0.0.1, and a Time to Live (TTL) of 64. The packet details further identify the traffic as belonging to TCP stream index 0 for the selected packet.

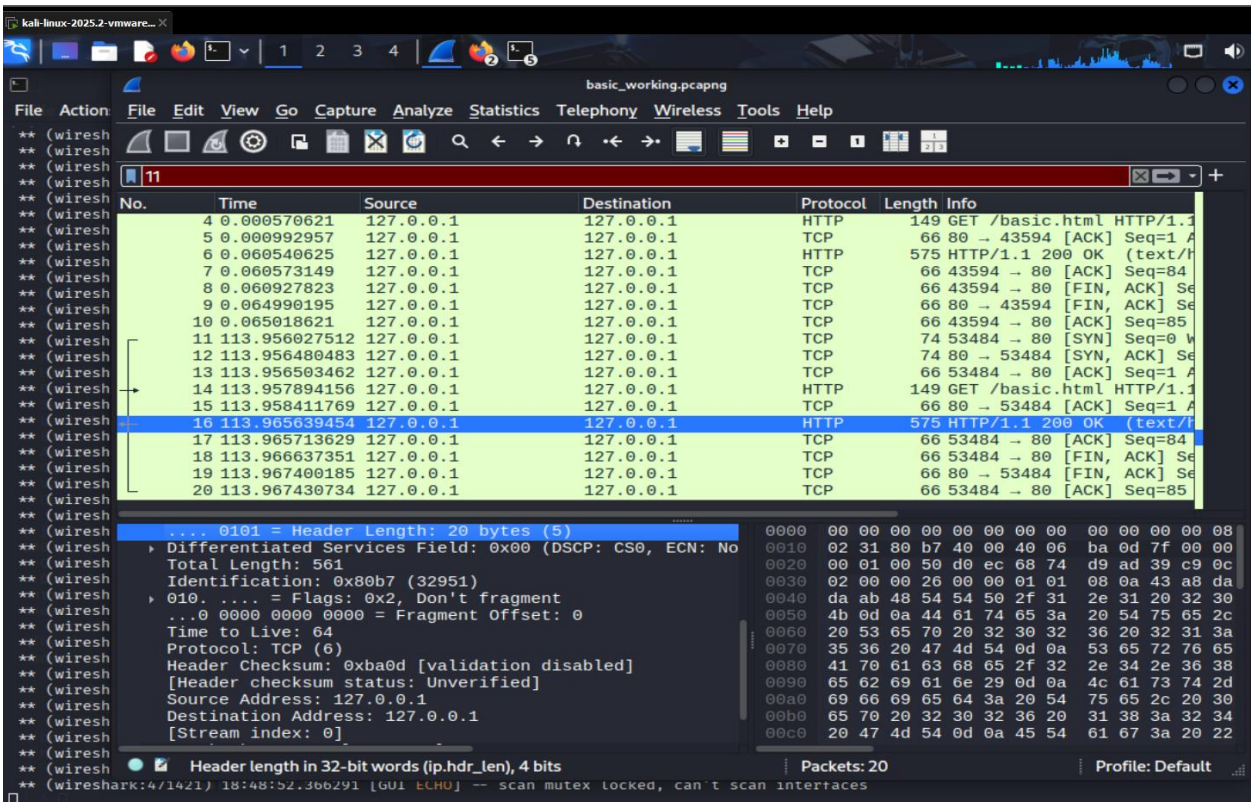
The packet list therefore establishes a direct relationship between the application-layer HTTP evidence and the underlying TCP transport. The request and response are not inferred solely from the curl output; they are also observable within the preserved packet capture, allowing the HTTP exchange to be independently examined and reconstructed in Wireshark.






kali-linux-2025.2-vmware...
basic_working.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
11
No. Time Source Destination Protocol Length Info
7 0.060573149 127.0.0.1 127.0.0.1 TCP 66 43594 → 80 [ACK] Seq=84
8 0.060927823 127.0.0.1 127.0.0.1 TCP 66 43594 → 80 [FIN, ACK] Se
9 0.064990195 127.0.0.1 127.0.0.1 TCP 66 80 → 43594 [FIN, ACK] Se
10 0.065018621 127.0.0.1 127.0.0.1 TCP 66 43594 → 80 [ACK] Seq=85
11 113.956027512 127.0.0.1 127.0.0.1 TCP 74 53484 → 80 [SYN] Seq=0 W
12 113.956480483 127.0.0.1 127.0.0.1 TCP 74 80 → 53484 [SYN, ACK] Se
13 113.956503462 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [ACK] Seq=1 A
14 113.957894156 127.0.0.1 127.0.0.1 HTTP 149 GET /basic.html HTTP/1.1
15 113.958411769 127.0.0.1 127.0.0.1 TCP 66 80 → 53484 [ACK] Seq=1 A
16 113.965639454 127.0.0.1 127.0.0.1 HTTP 575 HTTP/1.1 200 OK (text/h
17 113.965713629 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [ACK] Seq=84
18 113.966637351 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [FIN, ACK] Se
19 113.967400185 127.0.0.1 127.0.0.1 TCP 66 80 → 53484 [FIN, ACK] Se
20 113.967430734 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [ACK] Seq=85
Frame 10: Packet, 66 bytes on wire (528 bits), 66 by
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00
Destination: 00:00:00_00:00:00 (00:00:00:00:00:00
.... ..0. .... = LG bit: Globall
....0 .... = IG bit: Individ
Source: 00:00:00_00:00:00 (00:00:00:00:00:00)
Type: IPv4 (0x0800)
[Stream index: 0]
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 1
Transmission Control Protocol, Src Port: 43594, Dst
0000 00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 0
0010 00 34 cc a5 40 00 40 06 70 1c 7f 00 00 01 7f 0
0020 00 01 aa 4a 00 50 93 29 1f 86 a0 cb d5 db 80 1
0030 02 00 fe 28 00 00 01 01 08 0a 43 a7 1d c6 43 a
0040 1d c6


kali-linux-2025.2-vmware...
basic_working.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
11
No. Time Source Destination Protocol Length Info
4 0.000570621 127.0.0.1 127.0.0.1 HTTP 149 GET /basic.html HTTP/1.1
5 0.000992957 127.0.0.1 127.0.0.1 TCP 66 80 → 43594 [ACK] Seq=1 A
6 0.060540625 127.0.0.1 127.0.0.1 HTTP 575 HTTP/1.1 200 OK (text/h
7 0.060573149 127.0.0.1 127.0.0.1 TCP 66 43594 → 80 [ACK] Seq=84
8 0.060927823 127.0.0.1 127.0.0.1 TCP 66 43594 → 80 [FIN, ACK] Se
9 0.064990195 127.0.0.1 127.0.0.1 TCP 66 80 → 43594 [FIN, ACK] Se
10 0.065018621 127.0.0.1 127.0.0.1 TCP 66 43594 → 80 [ACK] Seq=85
11 113.956027512 127.0.0.1 127.0.0.1 TCP 74 53484 → 80 [SYN] Seq=0 W
12 113.956480483 127.0.0.1 127.0.0.1 TCP 74 80 → 53484 [SYN, ACK] Se
13 113.956503462 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [ACK] Seq=1 A
14 113.957894156 127.0.0.1 127.0.0.1 HTTP 149 GET /basic.html HTTP/1.1
15 113.958411769 127.0.0.1 127.0.0.1 TCP 66 80 → 53484 [ACK] Seq=1 A
16 113.965639454 127.0.0.1 127.0.0.1 HTTP 575 HTTP/1.1 200 OK (text/h
17 113.965713629 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [ACK] Seq=84
18 113.966637351 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [FIN, ACK] Se
19 113.967400185 127.0.0.1 127.0.0.1 TCP 66 80 → 53484 [FIN, ACK] Se
20 113.967430734 127.0.0.1 127.0.0.1 TCP 66 53484 → 80 [ACK] Seq=85
Frame 14: Packet, 149 bytes on wire (1192 bits), 149 byte
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00),
Destination: 00:00:00_00:00:00 (00:00:00:00:00:00)
.... ..0. .... = LG bit: Globally uniq
....0 .... = IG bit: Individual ad
Source: 00:00:00_00:00:00 (00:00:00:00:00:00)
.... ..0. .... = LG bit: Globally uniq
....0 .... = IG bit: Individual ad
Type: IPv4 (0x0800)
[Stream index: 0]
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
0000 00 00 00 00 00 00 00 00 00 00 00 00 08
0010 00 87 33 fa 40 00 40 06 08 75 7f 00 00
0020 00 01 d0 ec 00 50 39 c9 0c 28 68 74 d9
0030 02 00 fe 7b 00 00 01 01 08 0a 43 a8 da
0040 da aa 47 45 54 20 2f 62 61 73 69 63 2e
0050 6c 20 48 54 54 50 2f 31 2e 31 0d 0a 48
0060 3a 20 31 32 37 2e 30 2e 30 2e 31 0d 0a
0070 72 2d 41 67 65 6e 74 3a 20 63 75 72 6c
0080 32 31 2e 30 0d 0a 41 63 63 65 70 74 3a
0090 2a 0d 0a 0d 0a
Header length in 32-bit words (ip.hdr_len), 4 bits
Packets: 20
Profile: Default



CONCLUSION

The practical successfully demonstrated the fundamental process of conducting a network forensic examination of plaintext HTTP traffic. A controlled HTTP environment was established using Apache on Kali Linux, and network communication between the local client and web server was captured for forensic examination. Analysis of the resulting traffic provides visibility into the TCP connection establishment, HTTP request and response exchange, transmitted content, and connection termination sequence.

The exercise also demonstrated the importance of maintaining evidence integrity throughout a forensic investigation. The original PCAPNG capture was preserved separately from the working copy, while SHA-256 hashing was used to verify that the evidence had not been altered during preservation. The ability to reconstruct the HTTP conversation and observe its headers and content illustrates the security weakness of transmitting information over unencrypted HTTP. Overall, the practical provided hands-on experience in packet capture, protocol analysis, evidence preservation, and forensic documentation, reinforcing the procedures required for reliable network forensic investigations.